FRONTEND · MODÜL 9

src/app/page.jsx
src/app/layout.jsx
src/app/[id]/page.jsx

Next.js'e Giriş

React'te bildiklerimizden yola çıkarak: routing, rendering, veri çekme ve SSG

App Router | Server Components | Data Fetching | SSG | Server Actions

GIRIŞ

React biliyoruz. Next.js ne ekliyor?

Konu	React (Vite)	Next.js
Routing	react-router-dom, elle tanımlanır	Klasör yapısı = URL (App Router)
Render	Tarayıcıda (CSR), boş HTML gelir	Varsayılan sunucuda (SSR), dolu HTML gelir
SEO	index.html'de tek, ortak metadata	Her sayfaya ayrı metadata
Veri	useEffect, TanStack Query, Redux	Bileşende await + yerleşik fetch cache
Backend	Ayrı sunucu veya Firebase	API route ve Server Action aynı projede
Optimizasyon	Elle yapılır	Image ve Font bileşenleri hazır

GIRIŞ

Bugünün yol haritası

01	**Routing**	App Router, Link, dinamik yollar, layout, metadata	`1-basics/`
02	**İleri Routing**	Parallel Routes, Intercepting Routes	`test-pages/`
03	**Optimizasyon**	Image, absolute import, Font	`2-advanced/`
04	**Rendering ve Veri**	CSR/SSR, Server ve Client, data fetching, cache	`2-advanced/`
05	**Navigation ve Form**	useRouter, redirect, usePathname, Server Actions	`2-advanced/`
06	**SSG**	Statik ve dinamik sayfa, generateStaticParams	`2-advanced/`

ROUTING · APP ROUTER

Klasör yapısı = URL yapısı

```
src/app/
├─ page.jsx                → /
├─ products/
│  ├─ page.jsx             → /products
│  └─ [id]/page.jsx        → /products/42
└─ profile/
   ├─ page.jsx             → /profile
   ├─ friends/page.jsx     → /profile/friends
   └─ edit/page.jsx        → /profile/edit
```

Tüm sayfalar src/app içinde

Her klasör URL'de bir parça olur.

Sayfa = page.jsx

İçinde page.jsx olmayan klasör sayfa değildir; bileşeni default export ederiz.

İç içe klasör = Nested route

<Route path> tanımı yok; Next.js dosyaları bulup router'a kendisi ekler.

ROUTING · LINK

Sayfalar arası geçiş: to yerine href

react-router-dom

```
<Link to="/products">
```

Router provider'ı içinde çalışır.

Next.js · next/link

```
<Link href="/products">
```

Ek kurulum yok; prop'un adı href.

app/products/page.jsx

```jsx
import Link from "next/link";

const Products = () => (
  <div>
    <Link href="/products/1">Ürün - 1</Link>
    <Link href={`/products/${id}`}>…</Link>
  </div>
);
```

<a> sayfayı baştan yükler; Link yüklemez, state korunur.

Görünen linklerin sayfaları önceden yüklenir (prefetch).

ROUTING · DYNAMIC ROUTES

:id yerine [id] klasörü

react-router-dom

`path="/products/:id"`

Parametre useParams() ile okunur.

Next.js

`products/[id]/page.jsx`

Parametre sayfaya otomatik gelen params prop'undan okunur.

app/products/[id]/page.jsx

```jsx
const Detail = async ({ params }) => {
  const { id } = await params;

  return <h1>{id} id'li Ürün Detayı</h1>;
};

export default Detail;
```

params bir Promise: önce await etmeliyiz. Bu yüzden sayfa bileşeni async.

ROUTING · CATCH-ALL SEGMENTS

Birden fazla parametre: [...slug]

URL	slug değeri
/docs/belge-1	["belge-1"]
/docs/belge-1/sayfa-10	["belge-1", "sayfa-10"]
/docs/belge-1/sayfa-10/satir-20	["belge-1", "sayfa-10", "satir-20"]

app/docs/[...slug]/page.jsx

```
const { slug } = await params;  // slug her zaman bir dizi
return <h2>{slug.join("/")}</h2>;
```

ROUTING · ROUTE GROUPS

Parantezli klasör URL'e girmez

```
src/app/
├ (auth)/
│  ├ template.jsx
│  ├ loading.jsx
│  ├ error.jsx
│  ├ login/page.jsx      → /login
│  └ register/page.jsx   → /register
└ products/page.jsx      → /products
```

`auth/login` → `/auth/login`

`(auth)/login` → `/login`

Neden gruplarız?

Çok sayıda route'u kategorize edip bulmayı kolaylaştırmak için.

Gruba özel dosyalar

template, loading ve error sadece bu gruptaki sayfaları etkiler.

Sayfaları saran ortak iskelet

layout.jsx

Header, footer, yetki kontrolü… Sayfa değişse de kalıcıdır, **state korunur.**

```jsx
const RootLayout = ({ children }) => (
  <html lang="tr">
    <body>
      <header>HEADER</header>
      <main>{children}</main>
      <footer>FOOTER</footer>
    </body>
  </html>
);
```

template.jsx

Layout ile aynı işi yapar; ama her geçişte yeniden oluşur, **state sıfırlanır.**

```jsx
"use client";

const Template = ({ children }) => {
  const [name, setName] = useState("");
  return (
    <aside>
      <h1>Selam, {name}</h1>
      <input onChange={…} />
    </aside> … {children}
  );
};
```

ROUTING · ÖZEL DOSYALAR

Dosya adı, görevini belirler

Dosya	Ne zaman devreye girer?	Not
page.jsx	URL'e gidildiğinde	Route'u oluşturur
layout.jsx	Klasördeki tüm sayfaları sarar	State korunur
template.jsx	Klasördeki tüm sayfaları sarar	Her geçişte state sıfırlanır
loading.jsx	Bileşen async işlemi beklerken	Klasöründeki sayfaları etkiler
error.jsx	Bileşen hata fırlattığında	"use client" zorunlu
not-found.jsx	Eşleşmeyen URL veya notFound()	404 sayfası

State ve if yerine özel dosyalar

React: her bileşende elle

```
const Login = () => {
  const [isLoading, setIsLoading] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => { /* fetch… */ }, []);

  if (isLoading) return <Loader />;
  if (error) return <Error msg={error} />;

  return <h1>Login Sayfası</h1>;
};
```

Next.js: dosya adıyla, otomatik

```
// (auth)/login/page.jsx
const Login = async () => {
  await delay(2500);
  return <h1>Login Sayfası</h1>;
};
// (auth)/loading.jsx → beklerken
const Loading = () => <h1>Yükleniyor…</h1>;
// (auth)/error.jsx → hata olunca
"use client";
const Error = ({ error, reset }) => …
```

Next.js'te state ve if yok: bileşen sadece asıl içeriği yazar, gerisini özel dosyalar halleder.

ROUTING · METADATA

Her sayfaya ayrı metadata = daha iyi SEO

Statik: metadata nesnesi

```jsx
// app/layout.jsx
export const metadata = {
  title: {
    default: "Getir",
    template: "%s | Getir",
  },
};

// app/products/page.jsx
export const metadata = { title: "Ürünler" };
// sekme başlığı: "Ürünler | Getir"
```

Dinamik: generateMetadata

```jsx
// app/products/[id]/page.jsx
export const generateMetadata =
  async ({ params }) ⇒ {
    const { id } = await params;

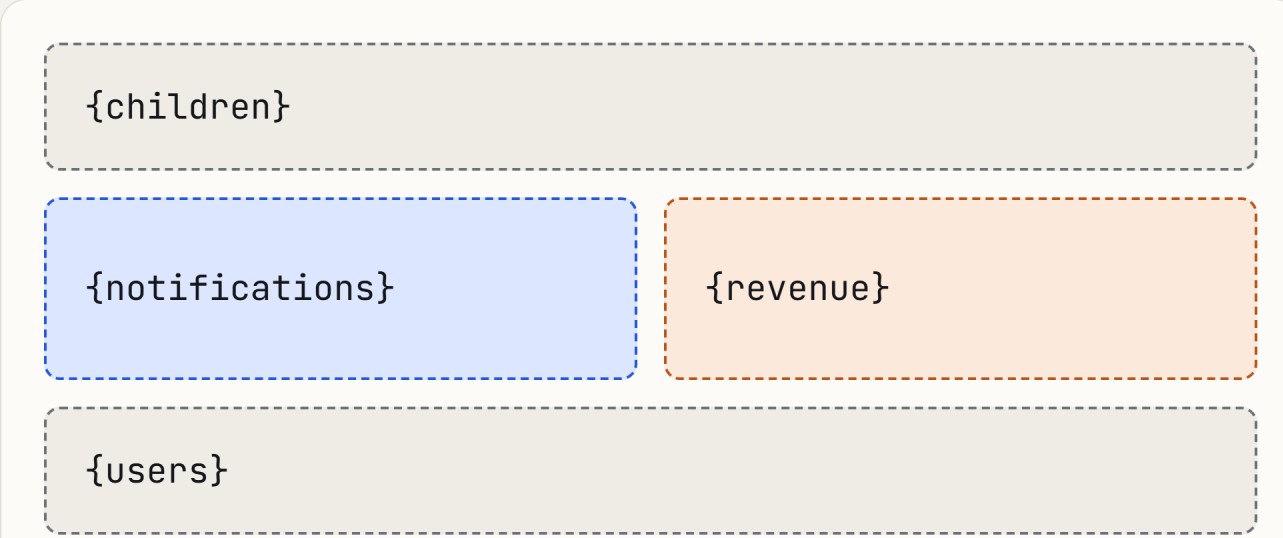
    // api'dan ürün bilgisi alınır…

    return {
      title: `${id} id'li ürün`,
    };
  };
```

Bir layout'ta birden fazla sayfa

```
dashboard/
├─ layout.jsx
├─ page.jsx          → children
├─ @notifications/   → notifications
├─ @revenue/         → revenue
└─ @users/           → users
   ├─ page.jsx
   └─ default.jsx

// layout.jsx
const Layout = ({
  children, notifications,
  revenue, users,
}) ⇒ ( … );
```



Ayrı route segmenti: kendi loading, error ve verisi; bağımsız render.

default.jsx: slot'ta eşleşen alt sayfa yoksa gösterilen yedek.

Sayfadan ayrılmadan, modal içinde sayfa

Listeden Link ile tıklanırsa

URL /wonders/3 olur, detay modal olarak açılır

`@modal/(..)wonders/[id]/page.jsx`

Sayfa yenilenir ya da link paylaşılırsa

Aynı URL tam sayfa detay olarak açılır

`wonders/[id]/page.jsx`

```
wonders/
├─ layout.jsx  ({ children, modal })
├─ page.jsx
├─ [id]/page.jsx
└─ @modal/
   ├─ default.jsx  → null
   └─ (..)wonders/[id]/page.jsx
```

Kural	Neyi yakalar?
(.)	Aynı seviyedeki route
(..)	Bir üst seviyedeki route
(..)(..)	İki üst seviyedeki route
(...)	app kökündeki route

Hazır gelen optimizasyonlar

Image

WebP'ye çevirir, lazy loading ve boyut optimizasyonu yapar.

```
import Image from "next/image";

<Image
  src={recipe.image}
  width={100}
  height={100}
  alt={recipe.name}
/>
```

Font

Google ya da yerel fontu CSS'e import etmeden yükler.

```
import { Oswald }
  from "next/font/google";
import LocalFont
  from "next/font/local";

const oswald = Oswald({
  subsets: ["latin"],
  variable: "--font-oswald",
});
```

Absolute import

@ işareti src'yi gösterir; dosya taşınsa da yol değişmez.

```
// relative
import { data } from
  "../../../../utils/constants";

// absolute
import { data } from
  "@/utils/constants";
```

RENDERING VE VERI

Next.js'te her bileşen varsayılan olarak Server Component'tir.

"use client" sadece hook ya da kullanıcı etkileşimi gerektiğinde.

RENDERING · CSR VE SSR

JavaScript nerede çalışıyor?

CSR · Client Side

01 Boş HTML ve JS dosyası indirilir

02 JS **kullanıcının cihazında** çalışır

03 HTML dolar, içerik görünür

```
gelen HTML: <div id="root"></div>
```

SSR · Server Side

01 JS **sunucuda** çalışır

02 HTML içeriği sunucuda oluşur

03 Dolu HTML indirilir, içerik hemen görünür

```
gelen HTML: <h1>Tarifler</h1><ul>…</ul>
```

SSR hem **SEO** (arama motoru dolu sayfayı okur) hem **performans** (içerik daha erken görünür) açısından daha iyi.

Server Component mi, Client Component mi?

Özellik	Server Component	Client Component
Nasıl tanımlanır?	Varsayılan, ek bir şey yok	En üst satıra “use client”
Nerede render olur?	Sunucuda (SSR)	Tarayıcıda (CSR)
useState, useEffect	Kullanılamaz	Kullanılabilir
onClick, onChange	Kullanılamaz	Kullanılabilir
Veri çekme	Bileşende doğrudan await	useEffect veya TanStack Query
SEO ve performans	Daha iyi	Daha zayıf

Kural: Sayfanın tamamını değil, sadece etkileşimli parçayı ayrı bir client component yap.

RENDERING · İÇ İÇE KULLANIM

Hangisi hangisinin içinde?

Aynı türden iki bileşen iç içe
Sorun yok

Server içinde Client
Sorun yok

Client içinde import edilen Server
Dikkat: server özelliklerini kaybeder, client'a dönüşür

Client'a children (HOC) olarak verilen Server
Server component olarak kalır

```
// about/page.jsx
<ClientComponent>
  <ServerComponent />
</ClientComponent>

// about/client-component.jsx
"use client";

const ClientComponent =
  ({ children }) => (
    <div>{children}</div>
  );
```

Üç state yerine tek await

Client: state + useEffect

```jsx
// recipes-client/page.jsx
"use client";
const [data, setData] = useState(null);
const [isLoading, setIsLoading] = useState(true);
const [error, setError] = useState(null);

useEffect(() => {
  fetch("https://dummyjson.com/recipes")
    .then((res) => res.json())
    .then((d) => setData(d.recipes))
    .catch((e) => setError(e.message))
    .finally(() => setIsLoading(false));
}, []);
```

Server: async bileşen

```jsx
// recipes-server/page.jsx
const RecipesServer = async () => {
  const data = await fetchRecipes();

  return data.recipes.map((r) => (
    <Recipe key={r.id} {...r} />
  ));
};

// yükleniyor → loading.jsx
// hata       → error.jsx
```

RENDERING · CACHE

fetch isteğinin cache ayarları

```
const res = await fetch(url, { cache: "no-store" });  // utils/service.js
```

Ayar	Davranış
cache: ”force-cache”	Veri bir kez çekilir, sonra hep cache’ten okunur
cache: ”no-store”	Her istekte API’ye yeniden gidilir
cache: ”no-cache”	Önce API’ye “veri değişti mi?” diye sorulur
next: { revalidate: 60 }	60 sn boyunca cache kullanılır, sonra yeniden çekilir

TanStack Query’de staleTime ile yaptığımızı burada fetch ayarı yapıyor; asenkron state için ek kütüphaneye daha az ihtiyaç var.

React Router'daki karşılıkları

react-router-dom	next/navigation	Nerede?
navigate("/x")	router.push("/x") (useRouter)	Client
navigate(-1), navigate(1)	router.back(), router.forward()	Client
<Navigate to="/login" />	redirect("/login")	Server
useLocation().pathname	usePathname()	Client
useParams()	useParams(), server'da params prop'u	Client
useSearchParams()	useSearchParams()	Client
path="*" ile 404 route'u	notFound()	İkisi de

Formu client yapmadan göndermek

React: state + onSubmit

```
const Form = () => {
  const [email, setEmail] = useState("");

  const handleSubmit = (e) => {
    e.preventDefault();
    api.post("/login", { email });
  };

  return (
    <form onSubmit={handleSubmit}>
      <input value={email} onChange={…} />
    </form>
  );
};
```

Next.js: action + formData

```
// app/form/page.jsx
const handleSubmitAction = async (formData) => {
  "use server";
  const email = formData.get("email");
  // api isteği veya veritabanı işlemi…
};

const Form = () => (
  <form action={handleSubmitAction}>
    <input name="email" type="email" />
    <button>Gönder</button>
  </form>
);
```

SSG · STATIC SITE GENERATION

HTML ne zaman hazırlanıyor?

STATIK SAYFA

Build anında

HTML, **next build** sırasında üretilip sunucuda saklanır. Kullanıcı gelince hazır sayfa anında sunulur.

Varsayılan: URL parametresi olmayan sayfalar

`/products · /recipes-server`

DINAMIK SAYFA

İstek anında

HTML, kullanıcı sayfaya girdiği anda sunucuda üretilir. İçerik her zaman güncel dir.

Varsayılan: URL parametresi olan sayfalar

`/products/[id] · /docs/[...slug]`

```
next build çıktısı:  ○ Static    ● SSG (generateStaticParams)    ƒ Dynamic
```

Varsayılanı değiştirmek

Statik → belli aralıkla yenile

Sayfa 60 saniyede bir yeniden oluşur

```
export const revalidate = 60;
```

Statik → tamamen dinamik

Her istekte yeniden oluşur

```
export const dynamic = "force-dynamic";
```

Dinamik → statik

Listedeki her id için build anında ayrı bir statik sayfa üretilir

```
// recipes-server/[id]/page.jsx
export const generateStaticParams = async () => {
  const data = await fetchRecipes();
  return data.recipes.map((i) => ({ id: String(i.id) }));
};
```

KAPANIŞ

Akılda kalsın

01 Klasör = route, sayfa = **page.jsx**, geçiş = **Link href**. Bekleme ve hata için loading.jsx ve error.jsx.

02 Dashboard için **Parallel**, modal için **Intercepting** routes; resim ve font için next/image, next/font.

03 Varsayılan **Server Component**. “use client” sadece hook ve etkileşim için.

04 Veriyi bileşende **await** et; formu **Server Action** ile gönder.

05 Parametresiz sayfa **statik**, parametreli sayfa **dinamik**; revalidate, dynamic ve generateStaticParams ile değiştir.

Sıradaki proje: 3-admin-dashboard · Next.js + TypeScript